

Media casting

Article • 10/20/2022

This article shows you how to cast media to remote devices from a Universal Windows app.

Built-in media casting with MediaPlayerElement

The simplest way to cast media from a Universal Windows app is to use the built-in casting capability of the [MediaPlayerElement](#) control.

To allow the user to open a video file to be played in the **MediaPlayerElement** control, add the following namespaces to your project.

C#

```
using Windows.Storage;
using Windows.Storage.Pickers;
using Windows.Storage.Streams;
using Windows.Media.Core;
```

In your app's XAML file, add a **MediaPlayerElement** and set [AreTransportControlsEnabled](#) to true.

XML

```
<MediaPlayerElement Name="mediaPlayerElement" MinHeight="100"
MaxWidth="600" HorizontalAlignment="Stretch"
AreTransportControlsEnabled="True"/>
```

Add a button to let the user initiate picking a file.

XML

```
<Button x:Name="openButton" Click="openButton_Click" Content="Open"/>
```

In the [Click](#) event handler for the button, create a new instance of the [FileOpenPicker](#), add video file types to the [FileTypeFilter](#) collection, and set the starting location to the user's videos library.

Call **PickSingleFileAsync** to launch the file picker dialog. When this method returns, the result is a **StorageFile** object representing the video file. Check to make sure the file isn't null, which it will be if the user cancels the picking operation. Call the file's **OpenAsync** method to get an **IRandomAccessStream** for the file. Finally, create a new **MediaSource** object from the selected file by calling **CreateFromStorageFile** and assign it to the **MediaPlayerElement** object's **Source** property to make the video file the video source for the control.

C#

```
private async void openButton_Click(object sender, RoutedEventArgs e)
{
    //Create a new picker
    FileOpenPicker filePicker = new FileOpenPicker();

    //Add filetype filters. In this case wmv and mp4.
    filePicker.FileTypeFilter.Add(".wmv");
    filePicker.FileTypeFilter.Add(".mp4");

    //Set picker start location to the video library
    filePicker.SuggestedStartLocation = PickerLocationId.VideosLibrary;

    //Retrieve file from picker
    StorageFile file = await filePicker.PickSingleFileAsync();

    //If we got a file, load it into the media element
    if (file != null)
    {
        mediaPlayerElement.Source = MediaSource.CreateFromStorageFile(file);
        mediaPlayerElement.MediaPlayer.Play();
    }
}
```

Once the video is loaded in the **MediaPlayerElement**, the user can simply press the casting button on the transport controls to launch a built-in dialog that allows them to choose a device to which the loaded media will be cast.



ⓘ Note

Starting with Windows 10, version 1607, it is recommended that you use the **MediaPlayer** class to play media items. The **MediaPlayerElement** is a lightweight XAML control that is used to render the content of a **MediaPlayer** in a XAML page. The **MediaElement** control continues to be supported for backwards compatibility. For more information on using **MediaPlayer** and **MediaPlayerElement** to play

media content, see [Play audio and video with MediaPlayer](#). For information on using [MediaSource](#) and related APIs to work with media content, see [Media items, playlists, and tracks](#).

Media casting with the CastingDevicePicker

A second way to cast media to a device is to use the [CastingDevicePicker](#). To use this class, include the [Windows.Media.Casting](#) namespace in your project.

C#

```
using Windows.Media.Casting;
```

Declare a member variable for the **CastingDevicePicker** object.

C#

```
CastingDevicePicker castingPicker;
```

When your page is initialized, create a new instance of the casting picker and set the [Filter](#) to [SupportsVideo](#) property to indicate that the casting devices listed by the picker should support video. Register a handler for the [CastingDeviceSelected](#) event, which is raised when the user picks a device for casting.

C#

```
//Initialize our picker object
castingPicker = new CastingDevicePicker();

//Set the picker to filter to video capable casting devices
castingPicker.Filter.SupportsVideo = true;

//Hook up device selected event
castingPicker.CastingDeviceSelected += CastingPicker_CastingDeviceSelected;
```

In your XAML file, add a button to allow the user to launch the picker.

XML

```
<Button x:Name="castPickerButton" Content="Cast Button"
Click="castPickerButton_Click"/>
```

In the **Click** event handler for the button, call **TransformToVisual** to get the transform of a UI element relative to another element. In this example, the transform is the position of the cast picker button relative to the visual root of the application window. Call the **Show** method of the **CastingDevicePicker** object to launch the casting picker dialog. Specify the location and dimensions of the cast picker button so that the system can make the dialog fly out from the button that the user pressed.

C#

```
private void castPickerButton_Click(object sender, RoutedEventArgs e)
{
    //Retrieve the location of the casting button
    GeneralTransform transform =
    castPickerButton.TransformToVisual(Window.Current.Content as UIElement);
    Point pt = transform.TransformPoint(new Point(0, 0));

    //Show the picker above our casting button
    castingPicker.Show(new Rect(pt.X, pt.Y, castPickerButton.ActualWidth,
    castPickerButton.ActualHeight),
        Windows.UI.Popups.Placement.Above);
}
```

In the **CastingDeviceSelected** event handler, call the **CreateCastingConnection** method of the **SelectedCastingDevice** property of the event args, which represents the casting device selected by the user. Register handlers for the **ErrorOccurred** and **StateChanged** events. Finally, call **RequestStartCastingAsync** to begin casting, passing in the result to the **MediaPlayerElement** control's **MediaPlayer** object's **GetAsCastingSource** method to specify that the media to be cast is the content of the **MediaPlayer** associated with the **MediaPlayerElement**.

ⓘ Note

The casting connection must be initiated on the UI thread. Since the **CastingDeviceSelected** is not called on the UI thread, you must place these calls inside a call to **CoreDispatcher.RunAsync** which causes them to be called on the UI thread.

C#

```
private async void CastingPicker_CastingDeviceSelected(CastingDevicePicker
sender, CastingDeviceSelectedEventArgs args)
{
    //Casting must occur from the UI thread. This dispatches the casting
    calls to the UI thread.
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
```

```

async () =>
{
    //Create a casting conneciton from our selected casting device
    CastingConnection connection =
args.SelectedCastingDevice.CreateCastingConnection();

    //Hook up the casting events
    connection.ErrorOccurred += Connection_ErrorOccurred;
    connection.StateChanged += Connection_StateChanged;

    //Cast the content loaded in the media element to the selected
casting device
    await
connection.RequestStartCastingAsync(mediaPlayerElement.MediaPlayer.GetAsCast
ingSource());
});
}

```

In the **ErrorOccurred** and **StateChanged** event handlers, you should update your UI to inform the user of the current casting status. These events are discussed in detail in the following section on creating a custom casting device picker.

C#

```

private async void Connection_StateChanged(CastingConnection sender, object
args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
() =>
    {
        ShowMessageToUser("Casting Connection State Changed: " +
sender.State);
    });
}

private async void Connection_ErrorOccurred(CastingConnection sender,
CastingConnectionErrorOccurredEventArgs args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
() =>
    {
        ShowMessageToUser("Casting Connection State Changed: " +
sender.State);
    });
}

```

Media casting with a custom device picker

The following section describes how to create your own casting device picker UI by enumerating the casting devices and initiating the connection from your code.

To enumerate the available casting devices, include the [Windows.Devices.Enumeration](#) namespace in your project.

C#

```
using Windows.Devices.Enumeration;
```

Add the following controls to your XAML page to implement the rudimentary UI for this example:

- A button to start the device watcher that looks for available casting devices.
- A [ProgressRing](#) control to provide feedback to the user that casting enumeration is ongoing.
- A [ListBox](#) to list the discovered casting devices. Define an [ItemTemplate](#) for the control so that we can assign the casting device objects directly to the control and still display the [FriendlyName](#) property.
- A button to allow the user to disconnect the casting device.

XML

```
<Button x:Name="startWatcherButton" Content="Watcher Button"
Click="startWatcherButton_Click"/>
<ProgressRing x:Name="watcherProgressRing" IsActive="False"/>
<ListBox x:Name="castingDevicesListBox" MaxWidth="300"
HorizontalAlignment="Left"
SelectionChanged="castingDevicesListBox_SelectionChanged">
    <!--Listbox content is bound to the FriendlyName field of our casting
devices-->
    <ListBox.ItemTemplate>
        <DataTemplate>
            <TextBlock Text="{Binding Path=FriendlyName}"/>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>
<Button x:Name="disconnectButton" Content="Disconnect"
Click="disconnectButton_Click" Visibility="Collapsed"/>
```

In your code behind, declare member variables for the [DeviceWatcher](#) and the [CastingConnection](#).

C#

```
DeviceWatcher deviceWatcher;
CastingConnection castingConnection;
```

In the **Click** handler for the *startWatcherButton*, first update the UI by disabling the button and making the progress ring active while device enumeration is ongoing. Clear the list box of casting devices.

Next, create a device watcher by calling [DeviceInformation.CreateWatcher](#). This method can be used to watch for many different types of devices. Specify that you want to watch for devices that support video casting by using the device selector string returned by [CastingDevice.GetDeviceSelector](#).

Finally, register event handlers for the [Added](#), [Removed](#), [EnumerationCompleted](#), and [Stopped](#) events.

C#

```
private void startWatcherButton_Click(object sender, RoutedEventArgs e)
{
    startWatcherButton.IsEnabled = false;
    watcherProgressRing.IsActive = true;

    castingDevicesListBox.Items.Clear();

    //Create our watcher and have it find casting devices capable of video
    casting
    deviceWatcher =
    DeviceInformation.CreateWatcher(CastingDevice.GetDeviceSelector(CastingPlayb
ackTypes.Video));

    //Register for watcher events
    deviceWatcher.Added += DeviceWatcher_Added;
    deviceWatcher.Removed += DeviceWatcher_Removed;
    deviceWatcher.EnumerationCompleted +=
    DeviceWatcher_EnumerationCompleted;
    deviceWatcher.Stopped += DeviceWatcher_Stopped;

    //Start the watcher
    deviceWatcher.Start();
}
```

The **Added** event is raised when a new device is discovered by the watcher. In the handler for this event, create a new [CastingDevice](#) object by calling [CastingDevice.FromIdAsync](#) and passing in the ID of the discovered casting device, which is contained in the **DeviceInformation** object passed into the handler.

Add the **CastingDevice** to the casting device **ListBox** so that the user can select it. Because of the [ItemTemplate](#) defined in the XAML, the [FriendlyName](#) property will be used as the item text for in the list box. Because this event handler is not called on the UI thread, you must update the UI from within a call to [CoreDispatcher.RunAsync](#).

C#

```
private async void DeviceWatcher_Added(DeviceWatcher sender,
DeviceInformation args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    async () =>
    {
        //Add each discovered device to our listbox
        CastingDevice addedDevice = await
CastingDevice.FromIdAsync(args.Id);
        castingDevicesListBox.Items.Add(addedDevice);
    });
}
```

The **Removed** event is raised when the watcher detects that a casting device is no longer present. Compare the ID property of the **Added** object passed into the handler to the ID of each **Added** in the list box's **Items** collection. If the ID matches, remove that object from the collection. Again, because the UI is being updated, this call must be made from within a **RunAsync** call.

C#

```
private async void DeviceWatcher_Removed(DeviceWatcher sender,
DeviceInformationUpdate args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        foreach (CastingDevice currentDevice in castingDevicesListBox.Items)
        {
            if (currentDevice.Id == args.Id)
            {
                castingDevicesListBox.Items.Remove(currentDevice);
            }
        }
    });
}
```

The **EnumerationCompleted** event is raised when the watcher has finished detecting devices. In the handler for this event, update the UI to let the user know that device enumeration has completed and stop the device watcher by calling **Stop**.

C#

```
private async void DeviceWatcher_EnumerationCompleted(DeviceWatcher sender,
object args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
```



```

() =>
{
    //If enumeration completes, update UI and transition watcher to the
    stopped state
    ShowMessageToUser("Watcher completed enumeration of devices");
    deviceWatcher.Stop();
});
}

```

The Stopped event is raised when the device watcher has finished stopping. In the handler for this event, stop the **ProgressRing** control and reenale the *startWatcherButton* so that the user can restart the device enumeration process.

```

C#

private async void DeviceWatcher_Stopped(DeviceWatcher sender, object args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        //Update UX when the watcher stops
        startWatcherButton.IsEnabled = true;
        watcherProgressRing.IsActive = false;
    });
}

```

When the user selects one of the casting devices from the list box, the **SelectionChanged** event is raised. It is within this handler that the casting connection will be created and casting will be started.

First, make sure the device watcher is stopped so that device enumeration doesn't interfere with media casting. Create a casting connection by calling **CreateCastingConnection** on the **CastingDevice** object selected by the user. Add event handlers for the **StateChanged** and **ErrorOccurred** events.

Start media casting by calling **RequestStartCastingAsync**, passing in the casting source returned by calling the **MediaPlayer** method **GetAsCastingSource**. Finally, make the disconnect button visible to allow the user to stop media casting.

```

C#

private async void castingDevicesListBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    if (castingDevicesListBox.SelectedItem != null)
    {
        //When a device is selected, first thing we do is stop the watcher
        so it's search doesn't conflict with streaming
    }
}

```

```

        if (deviceWatcher.Status != DeviceWatcherStatus.Stopped)
        {
            deviceWatcher.Stop();
        }

        //Create a new casting connection to the device that's been selected
        castingConnection =
        ((CastingDevice)castingDevicesListBox.SelectedItem).CreateCastingConnection(
        );

        //Register for events
        castingConnection.ErrorOccurred += Connection_ErrorOccurred;
        castingConnection.StateChanged += Connection_StateChanged;

        //Cast the loaded video to the selected casting device.
        await
        castingConnection.RequestStartCastingAsync(mediaPlayerElement.MediaPlayer.Ge
        tAsCastingSource());
        disconnectButton.Visibility = Visibility.Visible;
    }
}

```

In the state changed handler, the action you take depends on the new state of the casting connection:

- If the state is **Connected** or **Rendering**, make sure the **ProgressRing** control is inactive and the disconnect button is visible.
- If the state is **Disconnected**, unselect the current casting device in the list box, make the **ProgressRing** control inactive, and hide the disconnect button.
- If the state is **Connecting**, make the **ProgressRing** control active and hide the disconnect button.
- If the state is **Disconnecting**, make the **ProgressRing** control active and hide the disconnect button.

C#

```

private async void Connection_StateChanged(CastingConnection sender, object
args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        //Update the UX based on the casting state
        if (sender.State == CastingConnectionState.Connected || sender.State
        == CastingConnectionState.Rendering)
        {
            disconnectButton.Visibility = Visibility.Visible;
            watcherProgressRing.IsActive = false;
        }
        else if (sender.State == CastingConnectionState.Disconnected)

```

```

    {
        disconnectButton.Visibility = Visibility.Collapsed;
        castingDevicesListBox.SelectedItem = null;
        watcherProgressRing.IsActive = false;
    }
    else if (sender.State == CastingConnectionState.Connecting)
    {
        disconnectButton.Visibility = Visibility.Collapsed;
        ShowMessageToUser("Connecting");
        watcherProgressRing.IsActive = true;
    }
    else
    {
        //Disconnecting is the remaining state
        disconnectButton.Visibility = Visibility.Collapsed;
        watcherProgressRing.IsActive = true;
    }
});
}

```

In the handler for the **ErrorOccurred** event, update your UI to let the user know that a casting error occurred and unselect the current **CastingDevice** object in the list box.

```

C#

private async void Connection_ErrorOccurred(CastingConnection sender,
CastingConnectionErrorOccurredEventArgs args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        //Clear the selection in the listbox on an error
        ShowMessageToUser("Casting Error: " + args.Message);
        castingDevicesListBox.SelectedItem = null;
    });
}

```

Finally, implement the handler for the disconnect button. Stop media casting and disconnect from the casting device by calling the **CastingConnection** object's **DisconnectAsync** method. This call must be dispatched to the UI thread by calling **CoreDispatcher.RunAsync**.

```

C#

private async void disconnectButton_Click(object sender, RoutedEventArgs e)
{
    if (castingConnection != null)
    {
        //When disconnect is clicked, the casting connection is
        disconnected. The video should return locally to the media element.
    }
}

```

```
        await castingConnection.DisconnectAsync();  
    }  
}
```

Enable audio playback from remote Bluetooth-connected devices

Article • 10/20/2022

This article shows you how to use [AudioPlaybackConnection](#) to enable Bluetooth-connected remote devices to play back audio on the local machine.

Starting with Windows 10, version 2004 remote audio sources can stream audio to Windows devices, enabling scenarios such as configuring a PC to behave like a Bluetooth speaker and allowing users to hear audio from their phone. The implementation uses the Bluetooth components in the OS to process incoming audio data and play it on the system's audio endpoints on the system such as built-in PC speakers or wired headphones. The enabling of the underlying Bluetooth A2DP sink is managed by apps, which are responsible for the end-user scenario, rather than by the system.

The [AudioPlaybackConnection](#) class is used to enable and disable connections from a remote device as well as to create the connection, allowing remote audio playback to begin.

Add a user interface

For the examples in this article, we will use the following simple XAML UI which defines **ListView** control to display available remote devices, a **TextBlock** to display connection status, and three buttons for enabling, disabling, and opening connections.

XML

```
<Grid x:Name="MainGrid" Loaded="MainGrid_Loaded">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <StackPanel Orientation="Horizontal">
        <TextBlock Text="Connection state: "/>
        <TextBlock x:Name="ConnectionState" Grid.Row="0"
Text="Disconnected."/>
    </StackPanel>
    <ListView x:Name="DeviceListView" ItemsSource="{x:Bind devices}"
Grid.Row="1">
        <ListView.ItemTemplate>
            <DataTemplate x:DataType="enumeration:DeviceInformation">
                <StackPanel Orientation="Horizontal" Margin="6">
```

```

        <SymbolIcon Symbol="Audio" Margin="0,0,12,0"/>
        <StackPanel>
            <TextBlock Text="{x:Bind Name}" FontWeight="Bold"/>
        </StackPanel>
    </StackPanel>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>
<StackPanel Orientation="Vertical" Grid.Row="2">
    <Button x:Name="EnableAudioPlaybackConnectionButton" Content="Enable
Audio Playback Connection"
Click="EnableAudioPlaybackConnectionButton_Click"/>
    <Button x:Name="ReleaseAudioPlaybackConnectionButton"
Content="Release Audio Playback Connection"
Click="ReleaseAudioPlaybackConnectionButton_Click"/>
    <Button x:Name="OpenAudioPlaybackConnectionButtonButton"
Content="Open Connection"
Click="OpenAudioPlaybackConnectionButtonButton_Click" IsEnabled="False"/>
</StackPanel>

</Grid>

```

Use DeviceWatcher to monitor for remote devices

The [DeviceWatcher](#) class allows you to detect connected devices. The [AudioPlaybackConnection.GetDeviceSelector](#) method returns a string that tells the device watcher what kinds of devices to watch for. Pass this string into the [DeviceWatcher](#) constructor.

The [DeviceWatcher.Added](#) event is raised for each device that is connected when the device watcher is started as well as for any device that is connected while the device watcher is running. The [DeviceWatcher.Removed](#) event is raised if a previously connected device disconnects.

Call [DeviceWatcher.Start](#) to begin watching for connected devices that support audio playback connections. In this example we will start the device manager when the main **Grid** control in the UI is loaded. For more information on using [DeviceWatcher](#), see [Enumerate Devices](#).

C#

```

private void MainGrid_Loaded(object sender, RoutedEventArgs e)
{
    audioPlaybackConnections = new Dictionary<string,
AudioPlaybackConnection>();
}

```

```

        // Start watching for paired Bluetooth devices.
        this.deviceWatcher =
        DeviceInformation.CreateWatcher(AudioPlaybackConnection.GetDeviceSelector())
        ;

        // Register event handlers before starting the watcher.
        this.deviceWatcher.Added += this.DeviceWatcher_Added;
        this.deviceWatcher.Removed += this.DeviceWatcher_Removed;

        this.deviceWatcher.Start();
    }

```

In the device watcher's **Added** event, each discovered device is represented by a [DeviceInformation](#) object. Add each discovered device to an observable collection that is bound to the **ListView** control in the UI.

C#

```

private ObservableCollection<Windows.Devices.Enumeration.DeviceInformation>
devices =
    new ObservableCollection<Windows.Devices.Enumeration.DeviceInformation>
    ();

```

C#

```

private async void DeviceWatcher_Added(DeviceWatcher sender,
DeviceInformation deviceInfo)
{
    // Collections bound to the UI are updated in the UI thread.
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, async () =>
    {
        this.devices.Add(deviceInfo);
    });
}

```

Enable and release audio playback connections

Before opening a connection with a device, the connection must be enabled. This informs the system that there is a new application that wants audio from the remote device to be played on the PC, but audio does not begin playing until the connection is opened, which is shown in a later step.

In the click handler for the **Enable Audio Playback Connection** button, get the device ID associated with the currently selected device in the **ListView** control. This example maintains a dictionary of **AudioPlaybackConnection** objects that have been enabled.

This method first checks to see if there is already an entry in the dictionary for the selected device. Next, the method attempts to create an **AudioPlaybackConnection** for the selected device by calling [TryCreateFromId](#) and passing in the selected device ID.

If the connection is successfully created, add the new **AudioPlaybackConnection** object to the app's dictionary, register a handler for the object's [StateChanged](#) event, and call [StartAsync](#) to notify the system that the new connection is enabled.

C#

```
private Dictionary<String, AudioPlaybackConnection>
audioPlaybackConnections;
```

C#

```
private async void EnableAudioPlaybackConnectionButton_Click(object sender,
RoutedEventArgs e)
{
    if (! (DeviceListView.SelectedItem is null))
    {
        var selectedDeviceId = (DeviceListView.SelectedItem as
DeviceInformation).Id;
        if (!this.audioPlaybackConnections.ContainsKey(selectedDeviceId))
        {
            // Create the audio playback connection from the selected device
            id and add it to the dictionary.
            // This will result in allowing incoming connections from the
            remote device.
            var playbackConnection =
AudioPlaybackConnection.TryCreateFromId(selectedDeviceId);

            if (playbackConnection != null)
            {
                // The device has an available audio playback connection.
                playbackConnection.StateChanged +=
this.AudioPlaybackConnection_ConnectionStateChanged;
                this.audioPlaybackConnections.Add(selectedDeviceId,
playbackConnection);
                await playbackConnection.StartAsync();
                OpenAudioPlaybackConnectionButton.IsEnabled = true;
            }
        }
    }
}
```

Open the audio playback connection

In the previous step, an audio playback connection was created, but sound does not begin playing until the connection is opened by calling [Open](#) or [OpenAsync](#). In the **Open Audio Playback Connection** button click handler, get the currently selected device and use the ID to retrieve the **AudioPlaybackConnection** from the app's dictionary of connections. Await a call to **OpenAsync** and check the **Status** value of the returned [AudioPlaybackConnectionOpenResultStatus](#) object to see if the connection was opened successfully and, if so, update the connection state text box.

C#

```
private async void OpenAudioPlaybackConnectionButton_Click(object sender, RoutedEventArgs e)
{
    var selectedDevice = (DeviceListView.SelectedItem as DeviceInformation).Id;
    AudioPlaybackConnection selectedConnection;

    if (this.audioPlaybackConnections.TryGetValue(selectedDevice, out selectedConnection))
    {
        if ((await selectedConnection.OpenAsync()).Status == AudioPlaybackConnectionOpenResultStatus.Success)
        {
            // Notify that the AudioPlaybackConnection is connected.
            ConnectionState.Text = "Connected";
        }
        else
        {
            // Notify that the connection attempt did not succeed.
            ConnectionState.Text = "Disconnected (attempt failed)";
        }
    }
}
```

Monitor audio playback connection state

The [AudioPlaybackConnection.ConnectionStateChanged](#) event is raised whenever the state of the connection changes. In this example, the handler for this event updates the status text box. Remember to update the UI inside a call to [Dispatcher.RunAsync](#) to make sure the update is made on the UI thread.

C#

```
private async void AudioPlaybackConnection_ConnectionStateChanged(AudioPlaybackConnection sender, object args)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
```

```

{
    if (sender.State == AudioPlaybackConnectionState.Closed)
    {
        ConnectionState.Text = "Disconnected";
    }
    else if (sender.State == AudioPlaybackConnectionState.Opened)
    {
        ConnectionState.Text = "Connected";
    }
    else
    {
        ConnectionState.Text = "Unknown";
    }
});
}

```

Release connections and handle removed devices

This example provides a **Release Audio Playback Connection** button to allow the user to release an audio playback connection. In the handler for this event, we get the currently selected device and use the device's ID to look up the **AudioPlaybackConnection** in the dictionary. Call **Dispose** to release the reference and free any associated resources and remove the connection from the dictionary.

C#

```

private void ReleaseAudioPlaybackConnectionButton_Click(object sender,
RoutedEventArgs e)
{
    // Check if an audio playback connection was already created for the
    // selected device Id. If it was then release its reference to deactivate it.
    // The underlying transport is deactivated when all references are
    // released.
    if (!(DeviceListView.SelectedItem is null))
    {
        var selectedDeviceId = (DeviceListView.SelectedItem as
DeviceInformation).Id;
        if (audioPlaybackConnections.ContainsKey(selectedDeviceId))
        {
            AudioPlaybackConnection connectionToRemove =
audioPlaybackConnections[selectedDeviceId];
            connectionToRemove.Dispose();
            this.audioPlaybackConnections.Remove(selectedDeviceId);

            // Notify that the media device has been deactivated.
            ConnectionState.Text = "Disconnected";
            OpenAudioPlaybackConnectionButton.IsEnabled = false;
        }
    }
}

```

```

    }
}
}

```

You should handle the case where a device is removed while a connection is enabled or open. To do this, implement a handler for the device watcher's [DeviceWatcher.Removed](#) event. First, the ID of the removed device is used to remove the device from the observable collection bound to the app's **ListView** control. Next, if a connection associated with this device is in the app's dictionary, **Dispose** is called to free the associated resources and then the connection is removed from the dictionary. All of this is done within a call to **Dispatcher.RunAsync** to make sure the UI updates are performed on the UI thread.

C#

```

private async void DeviceWatcher_Removed(DeviceWatcher sender,
DeviceInformationUpdate deviceInfoUpdate)
{
    // Collections bound to the UI are updated in the UI thread.
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        // Find the device for the given id and remove it from the list.
        foreach (DeviceInformation device in this.devices)
        {
            if (device.Id == deviceInfoUpdate.Id)
            {
                this.devices.Remove(device);
                break;
            }
        }

        if (audioPlaybackConnections.ContainsKey(deviceInfoUpdate.Id))
        {
            AudioPlaybackConnection connectionToRemove =
audioPlaybackConnections[deviceInfoUpdate.Id];
            connectionToRemove.Dispose();
            this.audioPlaybackConnections.Remove(deviceInfoUpdate.Id);
        }
    });
}

```

Related topics

[Media Playback](#)

PlayReady DRM

Article • 10/27/2022

This topic describes how to add PlayReady protected media content to your Universal Windows Platform (UWP) app.

PlayReady DRM enables developers to create UWP apps capable of providing PlayReady content to the user while enforcing the access rules defined by the content provider. This section describes changes made to Microsoft PlayReady DRM for Windows 10 and how to modify your PlayReady UWP app to support the changes made from the previous Windows 8.1 version to the Windows 10 version.

 [Expand table](#)

Topic	Description
Hardware DRM	This topic provides an overview of how to add PlayReady hardware-based digital rights management (DRM) to your UWP app.
Adaptive streaming with PlayReady	This article describes how to add adaptive streaming of multimedia content with Microsoft PlayReady content protection to a Universal Windows Platform (UWP) app. This feature currently supports playback of Http Live Streaming (HLS) and Dynamic Streaming over HTTP (DASH) content.

What's new in PlayReady DRM

The following list describes the new features and changes made to PlayReady DRM for Windows 10.

- Added hardware digital rights management (HWDRM).

Hardware-based content protection support enables secure playback of high definition (HD) and ultra-high definition (UHD) content on multiple device platforms. Key material (including private keys, content keys, and any other key material used to derive or unlock said keys), and decrypted compressed and uncompressed video samples are protected by leveraging hardware security. When Hardware DRM is being used, neither unknown enabler (play to unknown / play to unknown with downres) has meaning as the HWDRM pipeline always knows the output being used. For more information, see [Hardware DRM](#).

- PlayReady is no longer an appX framework component, but instead is an in-box operating system component. The namespace was changed from

Microsoft.Media.PlayReadyClient to [Windows.Media.Protection.PlayReady](#).

- The following headers defining the PlayReady error codes are now part of the Windows Software Development Kit (SDK):

Windows.Media.Protection.PlayReadyErrors.h and

Windows.Media.Protection.PlayReadyResults.h.

- Provides proactive acquisition of non-persistent licenses.

Previous versions of PlayReady DRM did not support proactive acquisition of non-persistent licenses. This capability has been added to this version. This can decrease the time to first frame. For more information, see [Proactively Acquire a Non-Persistent License Before Playback](#).

- Provides acquisition of multiple licenses in one message.

Allows the client app to acquire multiple non-persistent licenses in one license acquisition message. This can decrease the time to first frame by acquiring licenses for multiple pieces of content while the user is still browsing your content library; this prevents a delay for license acquisition when the user selects the content to play. In addition, it allows audio and video streams to be encrypted to separate keys by enabling a content header that includes multiple key identifiers (KIDs); this enables a single license acquisition to acquire all licenses for all streams within a content file instead of having to use custom logic and multiple license acquisition requests to achieve the same result.

- Added real time expiration support, or limited duration license (LDL).

Provides the ability to set real-time expiration on licenses and smoothly transition from an expiring license to another (valid) license in the middle of playback. When combined with acquisition of multiple licenses in one message, this allows an app to acquire several LDLs asynchronously while the user is still browsing the content library and only acquire a longer duration license once the user has selected content to playback. Playback will then start more quickly (because a license is already available) and, since the app will have acquired a longer duration license by the time the LDL expires, smoothly continue playback to the end of the content without interruption.

- Added non-persistent license chains.
- Added support for time-based restrictions (including expiration, expire after first play, and real time expiration) on non-persistent licenses.
- Added HDCP Type 1 (version 2.2 on Windows 10) policy support.

See [Things to Consider](#) for more information.

- Miracast is now implicit as an output.
- Added secure stop.

Secure stop provides the means for a PlayReady device to confidently assert to a media streaming service that media playback has stopped for any given piece of content. This capability ensures your media streaming services provide accurate enforcement and reporting of usage limitations on different devices for a given account.

- Added audio and video license separation.

Separate tracks prevent video from being decoded as audio; enabling more robust content protection. Emerging standards are requiring separate keys for audio and visual tracks.

- Added MaxResDecode.

This feature was added to limit playback of content to a maximum resolution even when in possession of a more capable key (but not a license). It supports cases where multiple stream sizes are encoded with a single key.

The following new interfaces, classes, and enumerations were added to PlayReady DRM:

- [IPlayReadyLicenseAcquisitionServiceRequest](#) interface
- [IPlayReadyLicenseSession](#) interface
- [IPlayReadySecureStopServiceRequest](#) interface
- [PlayReadyLicenseSession](#) class
- [PlayReadySecureStopIterable](#) class
- [PlayReadySecureStopIterator](#) class
- [PlayReadyHardwareDRMFeatures](#) enumerator

A new sample has been created to demonstrate how to use the new features of PlayReady DRM. The sample can be downloaded from the [Code Samples Browser](#).

Things to consider

- PlayReady DRM now supports HDCP Type 1 (supported in HDCP version 2.1 or later). PlayReady carries an HDCP Type Restriction policy in the license for the device to enforce. On Windows 10, this policy will enforce that HDCP 2.2 or later is engaged. This feature can be enabled in your PlayReady Server v3.0 SDK license (the server controls this policy in the license using the HDCP Type Restriction

GUID). For more information, see the [PlayReady Compliance and Robustness Rules](#).

- Windows Media Video (also known as VC-1) is not supported in hardware DRM (see [Override Hardware DRM](#)).
- PlayReady DRM now supports the High Efficiency Video Coding (HEVC /H.265) video compression standard. To support HEVC, your app must use Common Encryption Scheme (CENC) version 2 content which includes leaving the content's slice headers in the clear. Refer to ISO/IEC 23001-7 Information technology -- MPEG systems technologies -- Part 7: Common encryption in ISO base media file format files (Spec version ISO/IEC 23001-7:2015 or later is required.) for more information. Microsoft also recommends using CENC version 2 for all HWDRM content. In addition, some hardware DRM will support HEVC and some will not (see [Override Hardware DRM](#)).
- To take advantage of certain new PlayReady 3.0 features (including, but not limited to, SL3000 for hardware-based clients, acquiring multiple non-persistent licenses in one license acquisition message, and time-based restrictions on non-persistent licenses), the PlayReady server is required to be the Microsoft PlayReady Server Software Development Kit v3.0.2769 Release version or later.
- Depending on the Output Protection Policy specified in the content license, media playback may fail for end users if their connected output does not support those requirements. The following table lists the set of common errors that occur as a result. For more information, see the [PlayReady Compliance and Robustness Rules](#).

 Expand table

Error	Value	Description
ERROR_GRAPHICS_OPM_OUTPUT_DOES_NOT_SUPPORT_HDCP	0xC0262513	The license's Output Protection Policy requires the monitor to engage HDCP, but HDCP was unable to be engaged.
MF_E_POLICY_UNSUPPORTED	0xC00D7159	The license's Output Protection Policy requires the monitor to engage HDCP

Error	Value	Description
		Type 1, but HDCP Type 1 was unable to be engaged.
DRM_E_TEE_OUTPUT_PROTECTION_REQUIREMENTS_NOT_MET	0x8004CD22	This error code only occurs when running under hardware DRM. The license's Output Protection Policy requires the monitor to engage HDCP or to reduce the content's effective resolution, but HDCP was unable to be engaged and the content's effective resolution could not be reduced because hardware DRM does not support reducing the content's resolution. Under software DRM, the content plays. See Considerations for Using Hardware DRM .
ERROR_GRAPHICS_OPM_NOT_SUPPORTED	0xC0262500	The graphics driver does not support Output Protection. For example, the monitor is connected through VGA or an appropriate

Error	Value	Description
		graphics driver for the digital output is not installed. In the latter case, the typical driver that is installed is the Microsoft Basic Display Adapter and installing an appropriate graphics driver will resolve the issue.

Output protection

The following section describes the behavior when using PlayReady DRM for Windows 10 with output protection policies in a PlayReady license.

PlayReady DRM supports output protection levels contained in the **Microsoft PlayReady Extensible Media Rights Specification**. This document can be found in the documentation pack that comes with PlayReady licensed products.

ⓘ Note

The allowed values for output protection levels that can be set by a licensing server are governed by the [PlayReady Compliance Rules](#) [↗].

PlayReady DRM allows playback of content with output protection policies only on output connectors as specified in the PlayReady Compliance Rules. For more information about output connector terms specified in the PlayReady Compliance Rules, see [Defined Terms for PlayReady Compliance and Robustness Rules](#) [↗].

This section focuses on output protection scenarios with PlayReady DRM for Windows 10 and PlayReady Hardware DRM for Windows 10, which is also available on some Windows clients. With PlayReady HWDRM, all output protections are enforced from within the Windows TEE implementation (see [Hardware DRM](#)). As a result, some behaviors differ from when using PlayReady SWDRM (software DRM):

- Support for Output Protection Level (OPL) for Uncompressed Digital Video 270: PlayReady HWDRM for Windows 10 doesn't support down-resolution and will

enforce that HDCP (High-bandwidth Digital Content Protection) is engaged. It is recommended that high definition content for HWDRM have an OPL greater than 270 (although it is not required). Additionally, you should set HDCP type restriction in the license (HDCP version 2.2 or later).

- Unlike SWDRM, with HWDRM, output protections are enforced on all monitors based on the least capable monitor. For example, if the user has two monitors connected where one supports HDCP and the other doesn't, playback will fail if the license requires HDCP even if the content is only being rendered on the monitor that supports HDCP. In SWDRM, content will play back as long as it's only being rendered on the monitor that supports HDCP.
- HWDRM is not guaranteed to be used by the client and secure unless the following conditions are met by the content keys and licenses:
 - The license used for the video content key must have a minimum security level of 3000.
 - Audio must be encrypted to a different content key than video, and the license used for audio must have a minimum security level of 2000. Alternatively, audio could be left in the clear.
- All SWDRM scenarios require that the minimum security level of the PlayReady license used for the audio and/or video content key is lower or equal to 2000.

Output protection levels

The following table outlines the mappings between various OPLs in the PlayReady license and how PlayReady DRM for Windows 10 enforces them.

Video

 Expand table

OPL	Compressed digital video	Uncompressed digital video	Analog TV
	Any	HDMI, DVI, DisplayPort, MHL	Component, Composite
100	N/A*	Passes content	Passes content
150		N/A*	Passes content when CGMS-A CopyNever is engaged or if CGMS-A can't be engaged

200			Passes content when CGMS-A CopyNever is engaged
250		Attempts to engage HDCP, but passes content regardless of result	N/A*
270		SWDRM: Attempts to engage HDCP. If HDCP fails to engage, the PC will constrain the effective resolution to 520,000 pixels per frame and pass the content	HWDRM: Passes content with HDCP. If HDCP fails to engage, playback to HDMI/DVI ports is blocked
300		**When HDCP type restriction is NOT defined:** Passes content with HDCP. If HDCP fails to engage, playback to HDMI/DVI ports is blocked. **When HDCP type restriction IS defined**: Passes content with HDCP 2.2 and content stream type set to 1. If HDCP fails to engage or content stream type can't be set to 1, playback to HDMI/DVI ports is blocked.	
400	Windows 10 never passes compressed digital video content to outputs, regardless of the subsequent OPL value. For more information about compressed digital video content, see the Compliance Rules for PlayReady Products .	N/A*	
500			

* Not all values for output protection levels can be set by a licensing server. For more information, see the [PlayReady Compliance Rules](#).

Audio

[Expand table](#)

OPL	Compressed digital audio	Uncompressed digital audio	Analog or USB audio
	HDMI, DisplayPort, MHL	HDMI, DisplayPort, MHL	Any
100	Passes content	Passes content	Passes content
150		Does NOT pass content	
200			
250	Passes content when HDCP is engaged on HDMI, DisplayPort, or MHL, or when SCMS is engaged and set to CopyNever		
300	Passes content when HDCP is engaged on HDMI, DisplayPort, or MHL		

Miracast

PlayReady DRM allows you to play content over Miracast output as soon as HDCP 2.0 or later is engaged. On Windows 10, however, Miracast is considered a *digital* output. For more information about Miracast scenarios, see the [PlayReady Compliance Rules](#). The following table outlines the mappings between various OPLs in the PlayReady license and how PlayReady DRM enforces them on Miracast outputs.

[Expand table](#)

OPL	Compressed digital audio	Uncompressed digital audio	Compressed digital video	Uncompressed digital video
100	Passes content when HDCP 2.0 or later is engaged. If it fails to engage, it does NOT pass content	Passes content when HDCP 2.0 or later is engaged. If it fails to engage, it does NOT pass content	N/A*	Passes content when HDCP 2.0 or later is engaged. If it fails to engage, it does NOT pass content
150		Does NOT pass content		N/A*
200				
250				Passes content when HDCP 2.0 or later is

270	N/A*		engaged. If it fails to engage, it does NOT pass content
300	Passes content when HDCP 2.0 or later is engaged. If it fails to engage, it does NOT pass content	Does NOT pass content	**When HDCP type restriction is NOT defined:** Passes content when HDCP 2.0 or later is engaged. If it fails to engage, it does NOT pass content. **When HDCP type restriction IS defined:** Passes content with HDCP 2.2 and content stream type set to 1. If HDCP fails to engage or content stream type can't be set to 1, it does NOT pass content.
400	N/A*	Windows 10 never passes compressed digital video content to outputs, regardless of the subsequent OPL value. For more information about compressed digital video content, see the Compliance Rules for PlayReady Products .	N/A*
500			

* Not all values for output protection levels can be set by a licensing server. For more information, see the [PlayReady Compliance Rules](#).

Additional explicit output restrictions

The following table describes the PlayReady DRM for Windows 10 implementation of explicit digital video output protection restrictions.

[Expand table](#)

Scenario	GUID	If...	Then...
----------	------	-------	---------

Maximum effective resolution decode size	9645E831-E01D-4FFF-8342-0A720E3E028F	Connected output is: digital video output, Miracast, HDMI, DVI, etc.	Passes content when constrained to: • (a) the width of the frame must be less than or equal to the maximum frame width in pixels and the height of the frame less than or equal to the maximum frame height in pixels, or • (b) the height of the frame must be less than or equal to the maximum frame width in pixels and the width of the frame less than or equal to the maximum frame height in pixels
HDCP type restriction	ABB2C6F1-E663-4625-A945-972D17B231E7	Connected output is: digital video output, Miracast, HDMI, DVI, etc.	Passes content with HDCP 2.2 and the content stream type set to 1. If HDCP 2.2 fails to engage or the content stream type can't be set to 1, it does NOT pass content. Uncompressed digital video output protection level of a value greater than or equal to 271 must also be specified

The following table describes the PlayReady DRM for Windows 10 implementation of explicit analog video output protection restrictions.

[Expand table](#)

Scenario	GUID	If...	Then...	
Analog computer monitor	D783A191-E083-4BAF-B2DA-E69F910B3772	Connected output is: VGA, DVI-analog, etc.	SWDRM: PC will constrain effective resolution to 520,000 epx per frame and pass content	HWDRM: Does NOT pass content
Analog component	811C5110-46C8-4C6E-8163-C0482A15D47E	Connected output is: component	SWDRM: PC will constrain effective resolution to 520,000 epx per frame and pass content	HWDRM: Does NOT pass content
Analog TV outputs	2098DE8D-7DDD-4BAB-	Analog TV OPL is less than 151	CGMS-A must be engaged	

	96C6-32EBB6FABEA3		
	225CD36F-F132-49EF-BA8C-C91EA28E4369	Analog TV OPL is less than 101 and license doesn't contain 2098DE8D-7DDD-4BAB-96C6-32EBB6FABEA3	CGMS-A engagement must be attempted, but content may play regardless of result
Automatic gain control and color stripe	C3FD11C6-F8B7-4D20-B008-1DB17D61F2DA	Passing content with resolution less than or equal to 520,000 px to analog TV output	Sets AGC only for component video and PAL mode when resolution is less than 520,000 px and sets AGC and color stripe information for NTSC when resolution is less than 520,000 px, according to table 3.5.7.3. in Compliance Rules
Digital-only output	760AE755-682A-41E0-B1B3-DCDF836A7306	Connected output is analog	Does not pass content

ⓘ Note

When using an adapter dongle such as "Mini DisplayPort to VGA" for playback, Windows 10 sees the output as digital video output, and can't enforce analog video policies.

The following table describes the PlayReady DRM for Windows 10 implementation that enables playing in other circumstances.

[Expand table](#)

Scenario	GUID	If...	Then...	
Unknown output	786627D8-C2A6-44BE-8F88-08AE255B01A7	If output can't reasonably be determined, or OPM can't be established with graphics driver	SWDRM: Passes content	HWDRM: Does NOT pass content
Unknown output with constriction	B621D91F-EDCC-4035-8D4B-DC71760D43E9	If output can't reasonably be determined, or OPM can't be established with graphics driver	SWDRM: PC will constrain effective resolution to 520,000 epx per	HWDRM: Does NOT pass content

Prerequisites

Before you begin creating your PlayReady-protected UWP app, the following software needs to be installed on your system:

- Windows 10.
- If you are compiling any of the samples for PlayReady DRM for UWP apps, you must use Microsoft Visual Studio 2015 or later to compile the samples. You can still use Microsoft Visual Studio 2013 to compile any of the samples from PlayReady DRM for Windows 8.1 Store Apps.

PlayReady UWP app migration guide

This section includes information on how to migrate your existing PlayReady Windows 8.x Store apps to Windows 10.

The namespace for PlayReady UWP apps on Windows 10 was changed from **Microsoft.Media.PlayReadyClient** to **Windows.Media.Protection.PlayReady**. This means that you will need to search and replace the old namespace with the new one in your code. You will still be referencing a winmd file. It is part of windows.media.winmd on the Windows 10 operating system. It is in windows.winmd as part of the TH's Windows SDK. For UWP, it's referenced in windows.foundation.universalappcontract.winmd.

To play back PlayReady-protected high definition (HD) content (1080p) and ultra-high definition (UHD) content, you will need to implement PlayReady hardware DRM. For information on how to implement PlayReady hardware DRM, see [Hardware DRM](#).

Some content is not supported in hardware DRM. For information on disabling hardware DRM and enabling software DRM, see [Override Hardware DRM](#).

Regarding the media protection manager, make sure your code has the following settings if it doesn't already:

CS

```
var mediaProtectionManager = new  
Windows.Media.Protection.MediaProtectionManager();  
  
mediaProtectionManager.Properties["Windows.Media.Protection.MediaProtectionS
```



```

systemId"] =
    '{F4637010-03C3-42CD-B932-B48ADF3A6A54}'
var cpsystems = new Windows.Foundation.Collections.PropertySet();
cpsystems["{F4637010-03C3-42CD-B932-B48ADF3A6A54}"] =

    "Windows.Media.Protection.PlayReady.PlayReadyWinRTTrustedInput";
mediaProtectionManager.Properties["Windows.Media.Protection.MediaProtectionS
ystemIdMapping"] = cpsystems;

mediaProtectionManager.Properties["Windows.Media.Protection.MediaProtectionC
ontainerGuid"] =
    "{9A04F079-9840-4286-AB92-E65BE0885F95}";

```

Proactively acquire a non-persistent license before playback

This section describes how to acquire non-persistent licenses proactively before playback begins.

In previous versions of PlayReady DRM, non-persistent licenses could only be acquired reactively during playback. In this version, you can acquire non-persistent licenses proactively before playback begins.

1. Proactively create a playback session where the non-persistent license can be stored. For example:

CS

```

var cpsystems = new Windows.Foundation.Collections.PropertySet();
cpsystems["{F4637010-03C3-42CD-B932-B48ADF3A6A54}"] =
    "Windows.Media.Protection.PlayReady.PlayReadyWinRTTrustedInput"; //
    PlayReady

var pmpSystemInfo = new Windows.Foundation.Collections.PropertySet();
pmpSystemInfo["Windows.Media.Protection.MediaProtectionSystemId"] = "
    {F4637010-03C3-42CD-B932-B48ADF3A6A54}";
pmpSystemInfo["Windows.Media.Protection.MediaProtectionSystemIdMapping"
] = cpsystems;
var pmpServer = new Windows.Media.Protection.MediaProtectionPMPServer(
    pmpSystemInfo );

```

2. Tie that playback session to the license acquisition class. For example:

CS

```

var licenseSessionProperties = new
Windows.Foundation.Collections.PropertySet();

```

```
licenseSessionProperties["Windows.Media.Protection.MediaProtectionPMPSe
rver"] = pmpServer;
var licenseSession = new
Windows.Media.Protection.PlayReady.PlayReadyLicenseSession(
licenseSessionProperties );
```

3. Create a license service request. For example:

CS

```
var laSR = licenseSession.CreateLAServiceRequest();
```

4. Perform the license acquisition using the service request created from step 3. The license will be stored in the playback session.

5. Tie the playback session to the media source for playback. For example:

CS

```
licenseSession.configureMediaProtectionManager( mediaProtectionManager
);
videoPlayer.msSetMediaProtectionManager( mediaProtectionManager );
```

Query for protection capabilities

Starting with Windows 10, version 1703, you can query HW DRM capabilities, such as decode codecs, resolution, and output protections (HDCP). Queries are performed with the [IsTypeSupported](#) method which takes a string representing the capabilities for which support is queried and a string specifying the key system to which the query applies. For a list of supported string values, see the API reference page for [IsTypeSupported](#). The following code example illustrates the usage of this method.

CS

```
using namespace Windows::Media::Protection;

ProtectionCapabilities^ sr = ref new ProtectionCapabilities();

ProtectionCapabilityResult result = sr->IsTypeSupported(
L"video/mp4; codecs=\"avc1.640028\"; features=\"decode-bpp=10,decode-
fps=29.97,decode-res-x=1920,decode-res-y=1080\"",
L"com.microsoft.playready");

switch (result)
{
    case ProtectionCapabilityResult::Probably:
```

```
// Queue up UHD HW DRM video
break;

case ProtectionCapabilityResult::Maybe:
// Check again after UI or poll for more info.
break;

case ProtectionCapabilityResult::NotSupported:
// Do not queue up UHD HW DRM video.
break;
}
```

Add secure stop

This section describes how to add secure stop to your UWP app.

Secure stop provides the means for a PlayReady device to confidently assert to a media streaming service that media playback has stopped for any given piece of content. This capability ensures your media streaming services provide accurate enforcement and reporting of usage limitations on different devices for a given account.

There are two primary scenarios for sending a secure stop challenge:

- When the media presentation stops because end of content was reached or when the user stopped the media presentation somewhere in the middle.
- When the previous session ends unexpectedly (for example, due to a system or app crash). The app will need to query, either at startup or shutdown, for any outstanding secure stop sessions and send challenge(s) separate from any other media playback.

For a sample implementation of secure stop, see the **securestop.cs** file in the PlayReady sample located at the [Code Sample Browser](#).

Use PlayReady DRM on Xbox One

To use PlayReady DRM in a UWP app on Xbox One, you will first need to register your [Partner Center](#) [↗](#) account that you're using to publish the app for authorization to use PlayReady. You can do this in one of two ways:

- Have your contact at Microsoft request permission.
- Apply for authorization by sending your Partner Center account and company name to pronxbox@microsoft.com.

Once you receive authorization, you'll need to add an additional `<DeviceCapability>` to the app manifest. You'll have to add this manually because there is currently no setting available in the App Manifest Designer. Follow these steps to configure it:

1. With the project open in Visual Studio, open the **Solution Explorer** and right-click **Package.appxmanifest**.
2. Select **Open With...**, choose **XML (Text) Editor**, and click **OK**.
3. Between the `<Capabilities>` tags, add the following `<DeviceCapability>`:

XML

```
<DeviceCapability Name="6a7e5907-885c-4bcb-b40a-073c067bd3d5" />
```

4. Save the file.

Finally, there is one last consideration when using PlayReady on Xbox One: on development kits, there is an SL150 limit (that is, they can't play SL2000 or SL3000 content). Retail devices are able to play content with higher security levels, but to test your app on a dev kit, you'll need to use SL150 content. You can test this content in one of the following ways:

- Use curated test content that requires SL150 licenses.
- Implement logic so that only certain authenticated test accounts are able to acquire SL150 licenses for certain content.

Use the approach that makes the most sense for your company and your product.

See also

- [Media playback](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Hardware DRM

Article • 10/20/2022

This topic provides an overview of how to add PlayReady hardware-based digital rights management (DRM) to your Universal Windows Platform (UWP) app.

ⓘ Note

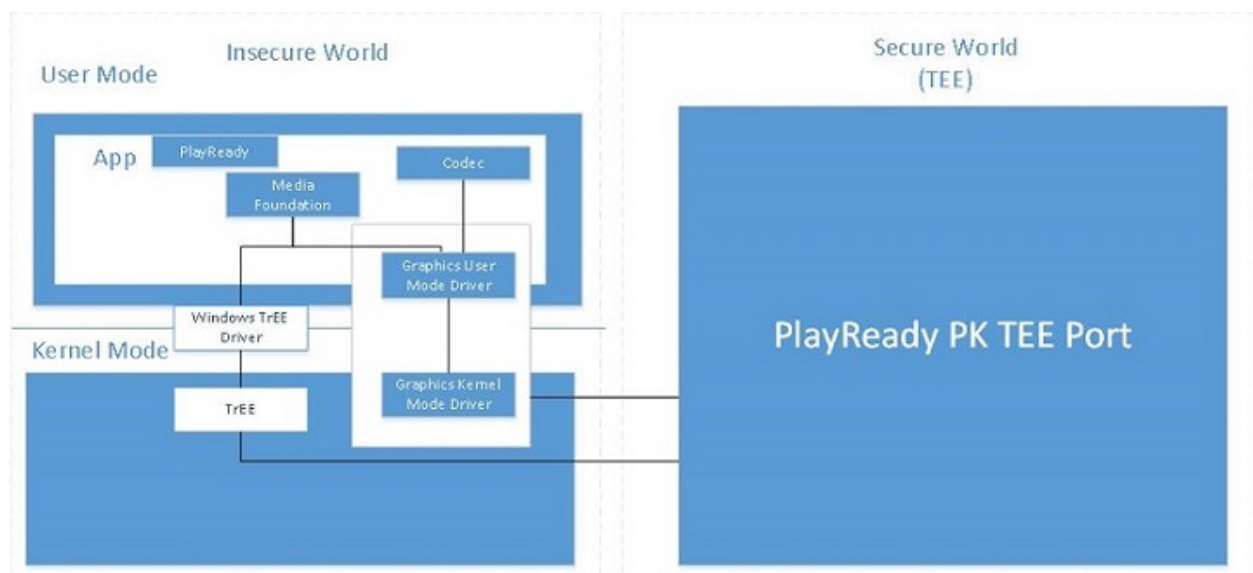
Hardware-based PlayReady DRM is supported on a multitude of devices, including both Windows and non-Windows devices such as TV sets, phones, and tablets. For a Windows device to support PlayReady Hardware DRM, it must be running Windows 10 and have a supported hardware configuration.

Increasingly, content providers are moving towards hardware-based protections for granting permission to play back full high value content in apps. Robust support for a hardware implementation of the cryptographic core has been added to PlayReady to meet this need. This support enables secure playback of high definition (1080p) and ultra-high definition (UHD) content on multiple device platforms. Key material (including private keys, content keys, and any other key material used to derive or unlock said keys), and decrypted compressed and uncompressed video samples are protected by leveraging hardware security.

Windows TEE implementation

This topic provides a brief overview of how Windows 10 implements the trusted execution environment (TEE).

The details of the Windows TEE implementation is out of scope for this document. However, a brief discussion of the difference between the standard porting kit TEE port and the Windows port will be beneficial. Windows implements the OEM proxy layer and transfers the serialized PRITEE functions calls to a user mode driver in the Windows Media Foundation subsystem. This will eventually get routed to either the Windows TrEE (Trusted Execution Environment) driver or the OEM's graphics driver. The details of either of these approaches is out of scope for this document. The following diagram shows the general component interaction for the Windows port. If you want to develop a Windows PlayReady TEE implementation, you can contact WMLA@Microsoft.com.



Considerations for using hardware DRM

This topic provides a brief list of items that should be considered when developing apps designed to use hardware DRM. As explained in [PlayReady DRM](#), with PlayReady HWDRM for Windows 10, all output protections are enforced from within the Windows TEE implementation, which has some consequences on output protection behaviors:

- **Support for output protection level (OPL) for uncompressed digital video 270:** PlayReady HWDRM for Windows 10 doesn't support down-resolution and will enforce that HDCP is engaged. We recommend that high definition content for HWDRM have an OPL greater than 270 (although it is not required). Additionally, we recommend that you set HDCP type restriction in the license (HDCP version 2.2 on Windows 10).
- **Unlike software DRM (SWDRM), output protections are enforced on all monitors based on the least capable monitor.** For example, if the user has two monitors connected where one of the monitors supports HDCP and the other does not, playback will fail if the license requires HDCP even if the content is only being rendered on the monitor that supports HDCP. In software DRM, content would play back as long as it is only being rendered on the monitor that supports HDCP.
- **HWDRM is not guaranteed to be used by the client and secure unless the following conditions are met** by the content keys and licenses:
 - The license used for the video content key must have a Minimum Security level property of 3000.
 - Audio must be encrypted to a different content key than video, and the license used for the audio must have a Minimum Security level property of 2000. Alternatively, audio could be left in the clear.

Additionally, you should take the following items into consideration when using HWDRM:

- Protected Media Process (PMP) is not supported.
- Windows Media Video (also known as VC-1) is not supported (see [Override hardware DRM](#)).
- Multiple graphics processing units (GPUs) are not supported for persistent licenses.

To handle persistent licenses on machines with multiple GPUs, consider the following scenario:

1. A customer buys a new machine with an integrated graphics card.
2. The customer uses an app that acquires persistent licenses while using hardware DRM.
3. The persistent license is now bound to that graphics card's hardware keys.
4. The customer then installs a new graphics card.
5. All licenses in the hashed data store (HDS) are bound to the integrated video card, but the customer now wants to play back protected content using the newly-installed graphics card.

To prevent playback from failing because the licenses can't be decrypted by the hardware, PlayReady uses a separate HDS for each graphics card that it encounters. This will cause PlayReady to attempt license acquisition for a piece of content where PlayReady would normally already have a license (that is, in the software DRM case or any case without a hardware change, PlayReady wouldn't need to reacquire a license). Therefore, if the app acquires a persistent license while using hardware DRM, your app needs to be able to handle the case where that license is effectively "lost" if the end user installs (or uninstalls) a graphics card. Because this is not a common scenario, you may decide to handle the support calls when the content no longer plays after a hardware change rather than figure out how to deal with a hardware change in the client/server code.

Override hardware DRM

This section describes how to override hardware DRM (HWDRM) if the content to be played back does not support hardware DRM.

By default, hardware DRM is used if the system supports it. However, some content is not supported in hardware DRM. One example of this is Cocktail content. Another example is any content that uses a video codec other than H.264 and HEVC. Another example is HEVC content, as some hardware DRM will support HEVC and some will not. Therefore, if you want to play a piece of content and hardware DRM doesn't support it on the system in question, you may want to opt out of hardware DRM.

The following example shows how to opt-out of hardware DRM. You only need to do this before you switch. Also, make sure you don't have any PlayReady object in memory, otherwise behavior is undefined.

JavaScript

```
var applicationData = Windows.Storage.ApplicationData.current;
var localSettings =
applicationData.localSettings.createContainer("PlayReady",
Windows.Storage.ApplicationDataCreateDisposition.always);
localSettings.values["SoftwareOverride"] = 1;
```

To switch back to hardware DRM, set the **SoftwareOverride** value to 0.

For every media playback, you need to set **MediaProtectionManager** to:

JavaScript

```
mediaProtectionManager.properties["Windows.Media.Protection.UseSoftwareProtectionLayer"] = true;
```

The best way to tell if you are in hardware DRM or software DRM is to look at C:\Users\
<username>\AppData\Local\Packages\<application name>\LocalCache\PlayReady*

- If there is an mspr.hds file, you are in software DRM.
- If you have another *.hds file, you are in hardware DRM.
- You can delete the entire PlayReady folder and retry your test as well.

Detect the type of hardware DRM

This section describes how to detect what type of hardware DRM is supported on the system.

You can use the [PlayReadyStatics.CheckSupportedHardware](#) method to determine whether the system supports a specific hardware DRM feature. For example:

C#

```
bool isFeatureSupported =
PlayReadyStatics.CheckSupportedHardware(PlayReadyHardwareDRMFeatures.HEVC);
```

The [PlayReadyHardwareDRMFeatures](#) enumeration contains the valid list of hardware DRM feature values that can be queried. To determine if hardware DRM is supported,

use the **HardwareDRM** member in the query. To determine if the hardware supports the High Efficiency Video Coding (HEVC)/H.265 codec, use the **HEVC** member in the query.

You can also use the [PlayReadyStatics.PlayReadyCertificateSecurityLevel](#) property to get the security level of the client certificate to determine if hardware DRM is supported. Unless the returned certificate security level is greater than or equal to 3000, either the client is not individualized or provisioned (in which case this property returns 0) or hardware DRM is not in use (in which case this property returns a value that is less than 3000).

Detecting support for AES128CBC hardware DRM

Starting with Windows 10, version 1709, you can detect support for AES128CBC hardware encryption on a device by calling [PlayReadyStatics.CheckSupportedHardware](#) and specifying the enumeration value [PlayReadyHardwareDRMFeatures.Aes128Cbc](#). On previous versions of Windows 10, specifying this value will cause an exception to be thrown. For this reason, you should check for the presence of the enumeration value by calling [ApiInformation.IsApiContractPresent](#) and specifying major contract version 5 before calling [CheckSupportedHardware](#).

C#

```
bool supportsAes128Cbc =  
ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContract", 5);  
  
if (supportsAes128Cbc)  
{  
    supportsAes128Cbc =  
    PlayReadyStatics.CheckSupportedHardware(PlayReadyHardwareDRMFeatures.Aes128Cbc);  
}
```

See also

- [PlayReady DRM](#)

Adaptive streaming with PlayReady

Article • 10/20/2022

This article describes how to add adaptive streaming of multimedia content with Microsoft PlayReady content protection to a Universal Windows Platform (UWP) app.

This feature currently supports playback of Dynamic streaming over HTTP (DASH) content.

HLS (Apple's HTTP Live Streaming) is not supported with PlayReady.

Smooth streaming is also currently not supported natively; however, PlayReady is extensible and by using additional code or libraries, PlayReady-protected Smooth streaming can be supported, leveraging software or even hardware DRM (digital rights management).

This article only deals with the aspects of adaptive streaming specific to PlayReady. For information about implementing adaptive streaming in general, see [Adaptive streaming](#).

This article uses code from the [Adaptive streaming sample](#) in Microsoft's **Windows-universal-samples** repository on GitHub. Scenario 4 deals with using adaptive streaming with PlayReady. You can download the repo in a ZIP file by navigating to the root level of the repository and selecting the **Download ZIP** button.

You will need the following **using** statements:

```
C#  
  
using LicenseRequest;  
using System;  
using System.Net.Http;  
using System.Net.Http.Headers;  
using System.Runtime.InteropServices;  
using System.Threading.Tasks;  
using Windows.Foundation.Collections;  
using Windows.MediaProtection;  
using Windows.MediaProtection.PlayReady;  
using Windows.Media.Streaming.Adaptive;  
using Windows.UI.Xaml.Controls;
```

The **LicenseRequest** namespace is from **CommonLicenseRequest.cs**, a PlayReady file provided by Microsoft to licensees.

You will need to declare a few global variables:

C#

```
private AdaptiveMediaSource ams = null;
private MediaProtectionManager protectionManager = null;
private string playReadyLicenseUrl = "";
private string playReadyChallengeCustomData = "";
```

You will also want to declare the following constant:

C#

```
private const uint MSPR_E_CONTENT_ENABLING_ACTION_REQUIRED = 0x8004B895;
```

Setting up the MediaProtectionManager

To add PlayReady content protection to your UWP app, you will need to set up a [MediaProtectionManager](#) object. You do this when initializing your [AdaptiveMediaSource](#) object.

The following code sets up a [MediaProtectionManager](#):

C#

```
private void SetUpProtectionManager(ref MediaElement mediaElement)
{
    protectionManager = new MediaProtectionManager();

    protectionManager.ComponentLoadFailed +=
        new
        ComponentLoadFailedEventHandler(ProtectionManager_ComponentLoadFailed);

    protectionManager.ServiceRequested +=
        new
        ServiceRequestedEventHandler(ProtectionManager_ServiceRequested);

    PropertySet cpSystems = new PropertySet();

    cpSystems.Add(
        "{F4637010-03C3-42CD-B932-B48ADF3A6A54}",
        "Windows.Media.Protection.PlayReady.PlayReadyWinRTTrustedInput");

    protectionManager.Properties.Add("Windows.Media.Protection.MediaProtectionSystemIdMapping", cpSystems);

    protectionManager.Properties.Add(
        "Windows.Media.Protection.MediaProtectionSystemId",
        "{F4637010-03C3-42CD-B932-B48ADF3A6A54}");
}
```

```

        protectionManager.Properties.Add(
            "Windows.Media.Protection.MediaProtectionContainerGuid",
            "{9A04F079-9840-4286-AB92-E65BE0885F95}");

        mediaElement.ProtectionManager = protectionManager;
    }

```

This code can simply be copied to your app, since it is mandatory for setting up content protection.

The [ComponentLoadFailed](#) event is fired when the load of binary data fails. We need to add an event handler to handle this, signaling that the load did not complete:

C#

```

private void ProtectionManager_ComponentLoadFailed(
    MediaProtectionManager sender,
    ComponentLoadFailedEventArgs e)
{
    e.Completion.Complete(false);
}

```

Similarly, we need to add an event handler for the [ServiceRequested](#) event, which fires when a service is requested. This code checks what kind of request it is, and responds appropriately:

C#

```

private async void ProtectionManager_ServiceRequested(
    MediaProtectionManager sender,
    ServiceRequestedEventArgs e)
{
    if (e.Request is PlayReadyIndividualizationServiceRequest)
    {
        PlayReadyIndividualizationServiceRequest indivRequest =
            e.Request as PlayReadyIndividualizationServiceRequest;

        bool bResultIndiv = await ReactiveIndivRequest(indivRequest,
            e.Completion);
    }
    else if (e.Request is PlayReadyLicenseAcquisitionServiceRequest)
    {
        PlayReadyLicenseAcquisitionServiceRequest licenseRequest =
            e.Request as PlayReadyLicenseAcquisitionServiceRequest;

        LicenseAcquisitionRequest(
            licenseRequest,
            e.Completion,
            playReadyLicenseUrl,
            playReadyChallengeCustomData);
    }
}

```

```
}  
}
```

Individualization service requests

The following code reactively makes a PlayReady individualization service request. We pass in the request as a parameter to the function. We surround the call in a try/catch block, and if there are no exceptions, we say the request completed successfully:

C#

```
async Task<bool> ReactiveIndivRequest(  
    PlayReadyIndividualizationServiceRequest IndivRequest,  
    MediaProtectionServiceCompletion CompletionNotifier)  
{  
    bool bResult = false;  
    Exception exception = null;  
  
    try  
    {  
        await IndivRequest.BeginServiceRequest();  
    }  
    catch (Exception ex)  
    {  
        exception = ex;  
    }  
    finally  
    {  
        if (exception == null)  
        {  
            bResult = true;  
        }  
        else  
        {  
            COMException comException = exception as COMException;  
            if (comException != null && comException.HResult ==  
MSPR_E_CONTENT_ENABLING_ACTION_REQUIRED)  
            {  
                IndivRequest.NextServiceRequest();  
            }  
        }  
    }  
  
    if (CompletionNotifier != null) CompletionNotifier.Complete(bResult);  
    return bResult;  
}
```

Alternatively, we may want to proactively make an individualization service request, in which case we call the following function in place of the code calling

ReactiveIndivRequest in ProtectionManager_ServiceRequested:

C#

```
async void ProActiveIndivRequest()
{
    PlayReadyIndividualizationServiceRequest indivRequest = new
    PlayReadyIndividualizationServiceRequest();
    bool bResultIndiv = await ReactiveIndivRequest(indivRequest, null);
}
```

License acquisition service requests

If instead the request was a [PlayReadyLicenseAcquisitionServiceRequest](#), we call the following function to request and acquire the PlayReady license. We tell the **MediaProtectionServiceCompletion** object that we passed in whether the request was successful or not, and we complete the request:

C#

```
async void LicenseAcquisitionRequest(
    PlayReadyLicenseAcquisitionServiceRequest licenseRequest,
    MediaProtectionServiceCompletion CompletionNotifier,
    string Url,
    string ChallengeCustomData)
{
    bool bResult = false;
    string ExceptionMessage = string.Empty;

    try
    {
        if (!string.IsNullOrEmpty(Url))
        {
            if (!string.IsNullOrEmpty(ChallengeCustomData))
            {
                System.Text.UTF8Encoding encoding = new
                System.Text.UTF8Encoding();
                byte[] b = encoding.GetBytes(ChallengeCustomData);
                licenseRequest.ChallengeCustomData =
                Convert.ToBase64String(b, 0, b.Length);
            }

            PlayReadySoapMessage soapMessage =
            licenseRequest.GenerateManualEnablingChallenge();

            byte[] messageBytes = soapMessage.GetMessageBody();
            HttpContent httpContent = new ByteArrayContent(messageBytes);

            IPropertySet propertySetHeaders = soapMessage.MessageHeaders;
```

```

        foreach (string strHeaderName in propertySetHeaders.Keys)
        {
            string strHeaderValue =
propertySetHeaders[strHeaderName].ToString();

            if (strHeaderName.Equals("Content-Type",
StringComparison.OrdinalIgnoreCase))
            {
                httpContent.Headers.ContentType =
MediaTypeHeaderValue.Parse(strHeaderValue);
            }
            else
            {
                httpContent.Headers.Add(strHeaderName.ToString(),
strHeaderValue);
            }
        }

        CommonLicenseRequest licenseAcquisition = new
CommonLicenseRequest();

        HttpContent responseHttpContent =
            await licenseAcquisition.AcquireLicense(new Uri(Url),
httpContent);

        if (responseHttpContent != null)
        {
            Exception exResult =
licenseRequest.ProcessManualEnablingResponse(
                                await
responseHttpContent.ReadAsByteArrayAsync());

            if (exResult != null)
            {
                throw exResult;
            }
            bResult = true;
        }
        else
        {
            ExceptionMessage = licenseAcquisition.GetLastErrorMessage();
        }
    }
    else
    {
        await licenseRequest.BeginServiceRequest();
        bResult = true;
    }
}
catch (Exception e)
{
    ExceptionMessage = e.Message;
}

```

```
CompletionNotifier.Complete(bResult);  
}
```

Initializing the AdaptiveMediaSource

Finally, you will need a function to initialize the [AdaptiveMediaSource](#), created from a given [Uri](#) and [MediaElement](#). The **Uri** should be the link to the media file (HLS or DASH); the **MediaElement** should be defined in your XAML.

C#

```
async private void InitializeAdaptiveMediaSource(System.Uri uri,  
MediaElement m)  
{  
    AdaptiveMediaSourceCreationResult result = await  
AdaptiveMediaSource.CreateFromUriAsync(uri);  
    if (result.Status == AdaptiveMediaSourceCreationStatus.Success)  
    {  
        ams = result.MediaSource;  
        SetUpProtectionManager(ref m);  
        m.SetMediaStreamSource(ams);  
    }  
    else  
    {  
        // Error handling  
    }  
}
```

You can call this function in whichever event handles the start of adaptive streaming; for example, in a button click event.

See also

- [PlayReady DRM](#)

PlayReady Encrypted Media Extension

Article • 10/20/2022

This section describes how to modify your PlayReady web app to support the changes made from the previous Windows 8.1 version to the Windows 10 version.

Using PlayReady media elements in Internet Explorer enables developers to create web apps capable of providing PlayReady content to the user while enforcing the access rules defined by the content provider. This section describes how to add PlayReady media elements to your existing web apps by using only HTML5 and JavaScript.

What's new in PlayReady Encrypted Media Extension

This section provides a list of changes made to the PlayReady Encrypted Media Extension (EME) to enable PlayReady content protection on Windows 10.

The following list describes the new features and changes made to PlayReady Encrypted Media Extension for Windows 10:

- Added hardware digital rights management (DRM).

Hardware-based content protection support enables secure playback of high definition (HD) and ultra-high definition (UHD) content on multiple device platforms. Key material (including private keys, content keys, and any other key material used to derive or unlock said keys), and decrypted compressed and uncompressed video samples are protected by leveraging hardware security.

- Provides proactive acquisition of non-persistent licenses.
- Provides acquisition of multiple licenses in one message.

You can either use a PlayReady object with multiple key identifiers (KeyIDs) as in Windows 8.1, or use [content decryption model data \(CDMData\)](#) with multiple KeyIDs.

ⓘ Note

In Windows 10, multiple key identifiers are supported under `<KeyID>` in CDMData.

- Added real time expiration support, or limited duration license (LDL).

Provides the ability to set real-time expiration on licenses.

- Added HDCP Type 1 (version 2.2) policy support.
- Miracast is now implicit as an output.
- Added secure stop.

Secure stop provides the means for a PlayReady device to confidently assert to a media streaming service that media playback has stopped for any given piece of content.

- Added audio and video license separation.

Separate tracks prevent video from being decoded as audio; enabling more robust content protection. Emerging standards are requiring separate keys for audio and visual tracks.

- Added MaxResDecode.

This feature was added to limit playback of content to a maximum resolution even when in possession of a more capable key (but not a license). It supports cases where multiple stream sizes are encoded with a single key.

Encrypted Media Extension support in PlayReady

This section describes the version of the W3C Encrypted Media Extension supported by PlayReady.

PlayReady for Web Apps is currently bound to the [W3C Encrypted Media Extension \(EME\) draft of May 10, 2013](#). This support will be changed to the updated EME specification in future versions of Windows.

Use hardware DRM

This section describes how your web app can use PlayReady hardware DRM, and how to disable hardware DRM if the protected content does not support it.

To use PlayReady hardware DRM, your JavaScript web app should use the `isTypeSupported` EME method with a key system identifier of

`com.microsoft.playready.hardware` to query for PlayReady hardware DRM support from the browser.

Occasionally, some content is not supported in hardware DRM. Cocktail content is never supported in hardware DRM; if you want to play cocktail content, you must opt out of hardware DRM. Some hardware DRM will support HEVC and some will not; if you want to play HEVC content and hardware DRM doesn't support it, you will want to opt out as well.

ⓘ Note

To determine whether HEVC content is supported, after instantiating `com.microsoft.playready`, use the [`PlayReadyStatics.CheckSupportedHardware`](#) method.

Add secure stop to your web app

This section describes how to add secure stop to your web app.

Secure stop provides the means for a PlayReady device to confidently assert to a media streaming service that media playback has stopped for any given piece of content. This capability ensures your media streaming services provide accurate enforcement and reporting of usage limitations on different devices for a given account.

There are two primary scenarios for sending a secure stop challenge:

- When the media presentation stops because end of content was reached or when the user stopped the media presentation somewhere in the middle.
- When the previous session ends unexpectedly (for example, due to a system or app crash). The app will need to query, either at startup or shutdown, for any outstanding secure stop sessions and send challenge(s) separate from any other media playback.

The following procedures describe how to set up secure stop for various scenarios.

To set up secure stop for a normal end of a presentation:

1. Register the **onEnded** event before playback starts.
2. The **onEnded** event handler needs to call `removeAttribute("src")` from the video/audio element object to set the source to **NULL** which will trigger the media foundation to tear down the topology, destroy the decryptor(s), and set the stop state.

3. You can start the secure stop CDM session inside the handler to send the secure stop challenge to the server to notify the playback has stopped at this time, but it can be done later as well.

To set up secure stop if the user navigates away from the page or closes down the tab or browser:

- No app action is required to record the stop state; it will be recorded for you.

To set up secure stop for custom page controls or user actions (such as custom navigation buttons or starting a new presentation before the current presentation completed):

- When custom user action occurs, the app needs to set the source to **NULL** which will trigger the media foundation to tear down the topology, destroy the decryptor(s), and set the stop state.

The following example demonstrates how to use secure stop in your web app:

JavaScript

```
// JavaScript source code

var g_prkey = null;
var g_keySession = null;
var g_fUseSpecificSecureStopSessionID = false;
var g_encodedMeteringCert = 'Base64 encoded of your metering cert (aka publisher cert)';

// Note: g_encodedLAsessionId is the CDM session ID of the proactive or reactive license acquisition
//       that we want to initiate the secure stop process.
var g_encodedLAsessionId = null;

function main()
{
    ...

    g_prkey = new MSMediaKeys("com.microsoft.playready");

    ...

    // add 'onended' event handler to the video element
    // Assume 'myvideo' is the ID of the video element
    var videoElement = document.getElementById("myvideo");
    videoElement.onended = function (e) {

        //
        // Calling removeAttribute("src") will set the source to null
        // which will trigger the MF to tear down the topology, destroy the
        // decryptor(s) and set the stop state. This is required in order
```

```

        // to set the stop state.
        //
        videoElement.removeAttribute("src");
        videoElement.load();

        onEndOfStream();
    };
}

function onEndOfStream()
{
    ...

    createSecureStopCDMSession();

    ...
}

function createSecureStopCDMSession()
{
    try{
        var targetMediaCodec = "video/mp4";
        var customData = "my custom data";

        var encodedSessionId = g_encodedLASessionId;
        if( !g_fUseSpecificSecureStopSessionID )
        {
            // Use "*" (wildcard) as the session ID to include all secure
            stop sessions
            // TODO: base64 encode "*" and place encoded result to
            encodedSessionId
        }

        var int8ArrayCDMdata = formatSecureStopCDMData( encodedSessionId,
        customData, g_encodedMeteringCert );
        var emptyArrayOfInitData = new Uint8Array();

        g_keySession = g_prkey.createSession(targetMediaCodec,
        emptyArrayOfInitData, int8ArrayCDMdata);

        addPlayreadyKeyEventHandler();

    } catch( e )
    {
        // TODO: Handle exception
    }
}

function addPlayreadyKeyEventHandler()
{
    // add 'keymessage' eventhandler
    g_keySession.addEventListener('mskeymessage', function (event) {

        // TODO: Get the keyMessage from event.message.buffer which contains
        the secure stop challenge
    });
}

```

```

//      The keyMessage format for the secure stop is similar to LA
as below:
//
//      <PlayReadyKeyMessage type="SecureStop" >
//          <SecureStop version="1.0" >
//              <Challenge encoding="base64encoded">
//                  secure stop challenge
//              </Challenge>
//              <HttpHeaders>
//                  <HTTPHeader>
//                      <name>Content-Type</name>
//                      <value>"content type data"</value>
//                  </HTTPHeader>
//                  <HTTPHeader>
//                      <name>SOAPAction</name>
//                      <value>soap action</value>
//                  </HTTPHeader>
//                  ....
//              </HttpHeaders>
//          </SecureStop>
//      </PlayReadyKeyMessage>

// TODO: send the secure stop challenge to a server that handles the
secure stop challenge

// TODO: Receive and response and call event.target.Update() to
process the response
});

// add 'keyerror' eventhandler
g_keySession.addEventListener('mskeyerror', function (event) {
    var session = event.target;

    ...

    session.close();
});

// add 'keyadded' eventhandler
g_keySession.addEventListener('mskeyadded', function (event) {

    var session = event.target;

    ...

    session.close();
});
}

/**
 * desc@ formatSecureStopCDMData
 * generate playready CDMData
 * CDMData is in xml format:
 * <PlayReadyCDMData type="SecureStop">
 *     <SecureStop version="1.0">

```

```

*      <SessionID>B64 encoded session ID</SessionID>
*      <CustomData>B64 encoded custom data</CustomData>
*      <ServerCert>B64 encoded server cert</ServerCert>
*      </SecureCert>
* </PlayReadyCDMData>
*/
function formatSecureStopCDMData(encodedSessionId, customData,
encodedPublisherCert)
{
    var encodedCustomData = null;

    // TODO: base64 encode the custom data and place the encoded result to
    encodedCustomData

    var CDMDataStr = "<PlayReadyCDMData type=\"SecureStop\">\" +
        "<SecureStop version=\"1.0\" >\" +
        "<SessionID>\" + encodedSessionId + "</SessionID>\" +
        "<CustomData>\" + encodedCustomData + "</CustomData>\" +
        "<ServerCert>\" + encodedPublisherCert + "</ServerCert>\"
+
        "</SecureStop></PlayReadyCDMData>\";

    var int8ArrayCDMdata = null

    // TODO: Convert CDMDataStr to Uint8 byte array and palce the converted
    result to int8ArrayCDMdata

    return int8ArrayCDMdata;
}

```

ⓘ Note

The secure stop data's `<SessionID>B64 encoded session ID</SessionID>` in the sample above can be an asterisk (*), which is a wild card for all the secure stop sessions recorded. That is, the **SessionID** tag can be a specific session, or a wild card (*) to select all the secure stop sessions.

Programming considerations for Encrypted Media Extension

This section lists the programming considerations that you should take into account when creating your PlayReady-enabled web app for Windows 10.

The **MSMediaKeys** and **MSMediaKeySession** objects created by your app must be kept alive until your app closes. One way of ensuring these objects stay alive is to assign them as global variables (the variables would become out of scope and subject to

garbage collection if declared as a local variable inside of a function). For example, the following sample assigns the variables *g_msMediaKeys* and *g_mediaKeySession* as global variables, which are then assigned to the **MSMediaKeys** and **MSMediaKeySession** objects in the function.

syntax

```
var g_msMediaKeys;
var g_mediaKeySession;

function foo() {
    ...
    g_msMediaKeys = new MSMediaKeys("com.microsoft.playready");
    ...
    g_mediaKeySession = g_msMediaKeys.createSession("video/mp4", intiData,
null);
    g_mediaKeySession.addEventListener(this.KEYMESSAGE_EVENT, function (e)
    {
        ...
        downloadPlayReadyKey(url, keyMessage, function (data)
        {
            g_mediaKeySession.update(data);
        });
    });
    g_mediaKeySession.addEventListener(this.KEYADDED_EVENT, function ()
    {
        ...
        g_mediaKeySession.close();
        g_mediaKeySession = null;
    });
}
```

For more information, see the [sample applications](#).

See also

- [PlayReady DRM](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Detect faces in images or videos

Article • 10/20/2022

This topic shows how to use the [FaceDetector](#) to detect faces in an image. The [FaceTracker](#) is optimized for tracking faces over time in a sequence of video frames.

For an alternative method of tracking faces using the [FaceDetectionEffect](#), see [Scene analysis for media capture](#).

The code in this article was adapted from the [Basic Face Detection](#) and [Basic Face Tracking](#) samples. You can download these samples to see the code used in context or to use the sample as a starting point for your own app.

Detect faces in a single image

The [FaceDetector](#) class allows you to detect one or more faces in a still image.

This example uses APIs from the following namespaces.

C#

```
using Windows.Storage;
using Windows.Storage.Pickers;
using Windows.Storage.Streams;
using Windows.Graphics.Imaging;
using Windows.Media.FaceAnalysis;
using Windows.UI.Xaml.Media.Imaging;
using Windows.UI.Xaml.Shapes;
```

Declare a class member variable for the [FaceDetector](#) object and for the list of [DetectedFace](#) objects that will be detected in the image.

C#

```
FaceDetector faceDetector;
IList<DetectedFace> detectedFaces;
```

Face detection operates on a [SoftwareBitmap](#) object which can be created in a variety of ways. In this example a [FileOpenPicker](#) is used to allow the user to pick an image file in which faces will be detected. For more information about working with software bitmaps, see [Imaging](#).

C#

```

FileOpenPicker photoPicker = new FileOpenPicker();
photoPicker.ViewMode = PickerViewMode.Thumbnail;
photoPicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
photoPicker.FileTypeFilter.Add(".jpg");
photoPicker.FileTypeFilter.Add(".jpeg");
photoPicker.FileTypeFilter.Add(".png");
photoPicker.FileTypeFilter.Add(".bmp");

StorageFile photoFile = await photoPicker.PickSingleFileAsync();
if (photoFile == null)
{
    return;
}

```

Use the [BitmapDecoder](#) class to decode the image file into a **SoftwareBitmap**. The face detection process is quicker with a smaller image and so you may want to scale the source image down to a smaller size. This can be performed during decoding by creating a [BitmapTransform](#) object, setting the [ScaledWidth](#) and [ScaledHeight](#) properties and passing it into the call to [GetSoftwareBitmapAsync](#), which returns the decoded and scaled **SoftwareBitmap**.

C#

```

IRandomAccessStream fileStream = await
photoFile.OpenAsync(FileAccessMode.Read);
BitmapDecoder decoder = await BitmapDecoder.CreateAsync(fileStream);

BitmapTransform transform = new BitmapTransform();
const float sourceImageHeightLimit = 1280;

if (decoder.PixelHeight > sourceImageHeightLimit)
{
    float scalingFactor = (float)sourceImageHeightLimit /
(float)decoder.PixelHeight;
    transform.ScaledWidth = (uint)Math.Floor(decoder.PixelWidth *
scalingFactor);
    transform.ScaledHeight = (uint)Math.Floor(decoder.PixelHeight *
scalingFactor);
}

SoftwareBitmap sourceBitmap = await
decoder.GetSoftwareBitmapAsync(decoder.BitmapPixelFormat,
BitmapAlphaMode.Premultiplied, transform,
ExifOrientationMode.IgnoreExifOrientation,
ColorManagementMode.DoNotColorManage);

```

In the current version, the **FaceDetector** class only supports images in Gray8 or Nv12. The **SoftwareBitmap** class provides the [Convert](#) method, which converts a bitmap from one format to another. This example converts the source image into the Gray8 pixel

format if it is not already in that format. If you want, you can use the [GetSupportedBitmapPixelFormat](#) and [IsBitmapPixelFormatSupported](#) methods to determine at runtime if a pixel format is supported, in case the set of supported formats is expanded in future versions.

C#

```
// Use FaceDetector.GetSupportedBitmapPixelFormat and
// IsBitmapPixelFormatSupported to dynamically
// determine supported formats
const BitmapPixelFormat faceDetectionPixelFormat = BitmapPixelFormat.Gray8;

SoftwareBitmap convertedBitmap;

if (sourceBitmap.BitmapPixelFormat != faceDetectionPixelFormat)
{
    convertedBitmap = SoftwareBitmap.Convert(sourceBitmap,
faceDetectionPixelFormat);
}
else
{
    convertedBitmap = sourceBitmap;
}
```

Instantiate the **FaceDetector** object by calling [CreateAsync](#) and then calling [DetectFacesAsync](#), passing in the bitmap that has been scaled to a reasonable size and converted to a supported pixel format. This method returns a list of [DetectedFace](#) objects. **ShowDetectedFaces** is a helper method, shown below, that draws squares around the faces in the image.

C#

```
if (faceDetector == null)
{
    faceDetector = await FaceDetector.CreateAsync();
}

detectedFaces = await faceDetector.DetectFacesAsync(convertedBitmap);
ShowDetectedFaces(sourceBitmap, detectedFaces);
```

Be sure to dispose of the objects that were created during the face detection process.

C#

```
sourceBitmap.Dispose();
fileStream.Dispose();
convertedBitmap.Dispose();
```

To display the image and draw boxes around the detected faces, add a **Canvas** element to your XAML page.

XML

```
<Canvas x:Name="VisualizationCanvas" Visibility="Visible" Grid.Row="0"
HorizontalAlignment="Stretch" VerticalAlignment="Stretch"/>
```

Define some member variables to style the squares that will be drawn.

C#

```
private readonly SolidColorBrush lineBrush = new
SolidColorBrush(Windows.UI.Colors.Yellow);
private readonly double lineThickness = 2.0;
private readonly SolidColorBrush fillBrush = new
SolidColorBrush(Windows.UI.Colors.Transparent);
```

In the **ShowDetectedFaces** helper method, a new **ImageBrush** is created and the source is set to a **SoftwareBitmapSource** created from the **SoftwareBitmap** representing the source image. The background of the XAML **Canvas** control is set to the image brush.

If the list of faces passed into the helper method isn't empty, loop through each face in the list and use the **FaceBox** property of the **DetectedFace** class to determine the position and size of the rectangle within the image that contains the face. Because the **Canvas** control is very likely to be a different size than the source image, you should multiply both the X and Y coordinates and the width and height of the **FaceBox** by a scaling value that is the ratio of the source image size to the actual size of the **Canvas** control.

C#

```
private async void ShowDetectedFaces(SoftwareBitmap sourceBitmap,
IList<DetectedFace> faces)
{
    ImageBrush brush = new ImageBrush();
    SoftwareBitmapSource bitmapSource = new SoftwareBitmapSource();
    await bitmapSource.SetBitmapAsync(sourceBitmap);
    brush.ImageSource = bitmapSource;
    brush.Stretch = Stretch.Fill;
    this.VisualizationCanvas.Background = brush;

    if (detectedFaces != null)
    {
        double widthScale = sourceBitmap.PixelWidth /
this.VisualizationCanvas.ActualWidth;
        double heightScale = sourceBitmap.PixelHeight /
this.VisualizationCanvas.ActualHeight;
```

```

        foreach (DetectedFace face in detectedFaces)
        {
            // Create a rectangle element for displaying the face box but
            // since we're using a Canvas
            // we must scale the rectangles according to the image's actual
            // size.
            // The original FaceBox values are saved in the Rectangle's Tag
            // field so we can update the
            // boxes when the Canvas is resized.
            Rectangle box = new Rectangle();
            box.Tag = face.FaceBox;
            box.Width = (uint)(face.FaceBox.Width / widthScale);
            box.Height = (uint)(face.FaceBox.Height / heightScale);
            box.Fill = this.fillBrush;
            box.Stroke = this.lineBrush;
            box.StrokeThickness = this.lineThickness;
            box.Margin = new Thickness((uint)(face.FaceBox.X / widthScale),
            (uint)(face.FaceBox.Y / heightScale), 0, 0);

            this.VisualizationCanvas.Children.Add(box);
        }
    }
}

```

Track faces in a sequence of frames

If you want to detect faces in video, it is more efficient to use the [FaceTracker](#) class rather than the [FaceDetector](#) class, although the implementation steps are very similar. The **FaceTracker** uses information about previously processed frames to optimize the detection process.

C#

```

using Windows.Media;
using System.Threading;
using Windows.System.Threading;

```

Declare a class variable for the **FaceTracker** object. This example uses a [ThreadPoolTimer](#) to initiate face tracking on a defined interval. A [SemaphoreSlim](#) is used to make sure that only one face tracking operation is running at a time.

C#

```

private FaceTracker faceTracker;
private ThreadPoolTimer frameProcessingTimer;
private SemaphoreSlim frameProcessingSemaphore = new SemaphoreSlim(1);

```